

Analyse comparative des méthodes d'intégration numérique

MGA 802 · Bouchra Berrissoul · Syndia Jean · Theophile Kevin Tsala Noah

1. Influence du nombre de segment sur la précision des méthodes

L'un des premiers phénomènes observés au cours des expériences concerne l'influence directe du nombre de segments (n) sur la précision des méthodes numériques. Pour chacune des approches implémentées, l'erreur absolue diminue progressivement lorsque le maillage devient plus fin (figure 1). Pour un polynôme cubique de référence ($p_1=1$, $p_2=-2$, $p_3=500$, $p_4=400$, $a=-2$, $b=3$), les méthodes se distinguent en deux groupes :

Les méthodes d'ordre 2 : rectangles et trapèzes

Les rectangles et les trapèzes convergent tous deux en $O(h^2)$, ce que les courbes log-log confirment avec une pente de exactement -2 (figure 2). Doubler n divise l'erreur par 4. Pour un polynôme cubique, il existe une relation théorique exacte entre les deux méthodes : l'erreur des rectangles (point milieu) vaut la moitié de l'erreur des trapèzes. Les rectangles sont donc systématiquement 2 fois plus précis que les trapèzes pour tout polynôme cubique, ce que les résultats expérimentaux confirment exactement (ratio Trap/Rect = 2.00 dans tous les cas testes).

Simpson : exactitude parfaite pour notre polynôme

La méthode de Simpson se distingue clairement des deux précédentes. Les résultats obtenus montrent que l'erreur devient extrêmement faible même pour un nombre limité de segments. Dans plusieurs cas de test, l'erreur atteint pratiquement la précision machine. Ce comportement est cohérent avec la théorie puisque la méthode de Simpson est exacte pour les polynômes de degré inférieur ou égal à 3, ce qui correspond exactement au type de fonction étudié dans le projet. En effet, la méthode de Simpson construit une parabole passant par les deux extrémités et le point milieu de chaque segment. Pour un polynôme de degré 3, cette parabole le reproduit exactement : l'erreur de troncature est rigoureusement nulle. Seul subsiste l'arrondi flottant ($\sim 1e-14$), indépendamment de n . SciPy Simpson donne des résultats numériquement quasi identiques. Malgré un nombre de segments contraint à être pair et une implémentation plus complexe, le gain de précision compense largement.

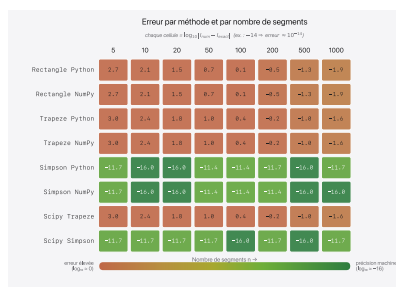


Fig.1 – Erreur par méthode

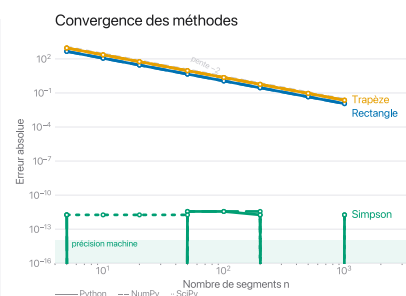


Fig.2 – Convergence

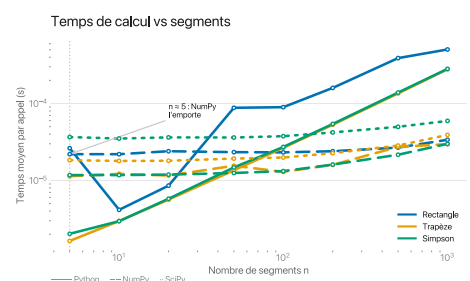


Fig.3 – Temps de calcul

Ainsi, contrairement aux méthodes précédentes qui réalisent uniquement une approximation géométrique locale, la méthode de Simpson exploite une interpolation quadratique qui reproduit parfaitement la structure du polynôme étudié.

Les graphiques de convergence (voir figure 2) obtenus permettent également de visualiser les ordres de grandeur entre les différentes méthodes. L'écart entre Simpson et les méthodes plus simples devient particulièrement visible lorsque le nombre de segments dépasse quelques centaines.

La précision dépend uniquement du choix algorithmique, pas de l'implémentation. Pour un polynôme cubique, Simpson surpasse les méthodes rectangle et trapèze.

2. Influence de la vectorisation sur la performances des implémentations

L'un des objectifs majeurs du projet était de démontrer l'intérêt de NumPy dans le calcul scientifique. Les implémentations réalisées mettent clairement en évidence la différence fondamentale entre une approche Python classique et une approche vectorisée.

Dans les versions Python pur, les calculs sont effectués élément par élément à l'intérieur de boucles for. Chaque itération implique une série d'opérations interprétées par Python, ce qui entraîne un coût important lorsque le nombre de segments devient élevé.

La version NumPy adopte une logique totalement différente. Tous les points sont générés simultanément grâce à `np.linspace`, puis les opérations mathématiques sont appliquées directement sur des tableaux complets. Cette vectorisation réduit fortement les surcoûts liés à l'interpréteur Python.

Les résultats temporels (figure 3) montrent une tendance très intéressante :

- Pour de petites valeurs de (n), Python pur peut parfois être plus rapide
- Lorsque (n) augmente, NumPy devient systématiquement plus rapide.

Ce phénomène s'explique par le coût fixe associé à la création des tableaux NumPy. Pour des tailles faibles, ce coût peut dépasser le gain apporté par la vectorisation. En revanche, lorsque le nombre de segments devient important, les optimisations internes de NumPy prennent largement l'avantage.

Les courbes de temps mettent particulièrement en évidence un point de bascule au-delà duquel les implémentations vectorisées deviennent nettement supérieures.

La performance dépend uniquement de l'implémentation. NumPy optimal pour $n > 40$. Python pur préférable pour $n < 35$. SciPy est plus rapide que Python pur mais plus lent que NumPy : son avantage est la robustesse et la lisibilité, pas la vitesse brute.

3. Analyse des expériences paramétriques

Trois expériences testent l'influence des paramètres. Nous prendrons $n = 100$ pour la valeur de référence.

Exp1 — Variation des coefficients p (bornes fixes $a=-2$, $b=3$)

Les résultats montrent que les polynômes présentant de fortes variations entraînent des erreurs plus importantes pour les méthodes simples. Lorsque les coefficients des termes de degré élevé augmentent, la courbure devient plus marquée et la méthode des rectangles perd rapidement en précision. Le cas linéaire (cas 1) donne $f'' = 0$, ce qui implique que toutes les méthodes deviennent exactes.

Cas	Polynome	Meilleure méthode et Erreur
Cas 1	$1 + x$	Toutes les méthodes ~ 0

Cas 2	$1 + x + 2x^2$	Simpson Numpy ~0
Cas 3	$1 + x + 10x^2 + 5x^3$	Simpson Python ~0
Cas 4	$1 + x + 100x^2 + 50x^3$	Scipy Simpson ~0
Cas 5	$1 + x + 500x^2 + 300x^3$	Scipy Simpson ~0

Exp2 — Variation des bornes a, b (coefficients p1=1, p2=-2, p3=500, p4=400 fixes)

Cette étude a montré que l'élargissement de l'intervalle augmente généralement l'erreur numérique puisque chaque segment couvre alors une portion plus importante de la fonction. Simpson reste exact dans tous les cas.

Cas	Intervalle	b-a	Meilleure méthode
Cas 1	[0, 0.5]	0.5	Simpson Python~0
Cas 2	[-2, 2]	4.0	Scipy Simpson ~4.547e-13
Cas 3	[-5, 5]	10	Scipy Simpson ~1.455e-11
Cas 4	[-10, 10]	20	Scipy Simpson ~0
Cas 5	[-20, 20]	40	Scipy Simpson ~4.657e-10

Exp3 — Variation simultanée de p et des bornes

Cas le plus général : les deux facteurs se combinent multiplicativement. Les cas avec forts coefficients cubiques et grands intervalles cumulent toutes les sources d'erreur pour Rectangle/Trapèze. Le cas A (linéaire, petit intervalle) rend toutes les méthodes exactes. Les mêmes tendances globales sont observées dans tous les cas.

Cas	Polynome & Intervalle	Meilleure méthode
Cas 1	$1 + x$ & [0, 0.5]	Toutes les méthodes ~0
Cas 2	$1 + x + 2x^2$ & [-2, 2]	Simpson Python~1.776e-15
Cas 3	$1 + x + 10x^2 + 5x^3$ & [-5, 5]	Simpson Python ~1.137e-13
Cas 4	$1 + x + 100x^2 + 50x^3$ & [-10, 10]	Simpson Python ~2.910e-11
Cas 5	$1 + x + 500x^2 + 300x^3$ & [-20, 20]	Scipy Simpson ~4.657e-10

Dans tous les cas, les mêmes tendances globales sont observées :

- Simpson reste la méthode la plus précise ;
- Pour les méthodes d'ordre 2, rectangles est systématiquement 2x plus précis que trapèzes.

4. Synthèse et recommandations

Deux décisions indépendantes

La précision est entièrement déterminée par la méthode mathématique : Python pur et NumPy produisent des erreurs identiques pour une même méthode. La performance est entièrement déterminée par l'implémentation : toutes les méthodes NumPy donnent ~29-33 us pour $n=1000$, toutes les méthodes Python donnent ~229-284 us, pour l'expérience 3. Précision et performance sont deux axes orthogonaux : choisir d'abord la méthode selon la précision requise, puis l'implémentation selon la valeur de n .

Méthode	Erreur moyenne	Temps moyenne (us)	Quand l'utiliser
Rectangle Python	5.468e-01	~229	$n < 35$, précision faible acceptable
Rectangle NumPy	5.468e-01	~30	$n > 50$, précision faible acceptable
Trapèze Python	1.094e+00	~249	$n < 35$, moins précis que Rectangle
Trapèze NumPy	1.094e+00	~29	$n > 50$, moins performant que Rectangle Numpy
Simpson Python	1.406e-09	~284	$n < 35$, plus rapide et précision comparable Numpy
Simpson NumPy	2.823e-10	~33	$n > 50$, plus rapide et plus précis que Python
SciPy Trapèze	1.094e+00	~39	Usage ponctuel, lisible mais lent
SciPy Simpson	7.451e-10	~61	Usage ponctuel, précision max

Limites de l'étude

Ces conclusions sont valables pour les polynômes de degré inférieur ou égal à 3. La précision exceptionnelle de Simpson est directement liée à cette structure : sur des fonctions oscillantes ou non polynomiales, les comportements seraient différents. Les mesures timeit dépendent de la machine et de la charge processeur. Pour $n > 10\,000$, la pression mémoire de NumPy pourrait modifier les courbes de performance observées. L'erreur de Simpson n'est pas rigoureusement $1e-14$ pour tous les cas : sur de grands intervalles avec de forts coefficients (EXP3 cas 5), les arrondis flottants peuvent atteindre $\sim 5e-10$ — négligeable face aux méthodes d'ordre 2.

Recommandation finale : Simpson NumPy pour tout $n > 40$. Cette combinaison offre la précision maximale ($\sim 1e-14$ pour les petits intervalles, $\sim 1e-10$ pour les grands) et la performance optimale ($\sim 33\ \mu s$ à $n=1000$), sans surcout par rapport aux autres variantes NumPy. Pour les méthodes d'ordre 2, préférer les rectangles aux trapèzes : même vitesse, deux fois plus précis.